

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 17
“Skema Pemrosesan Sekuensial”



DISUSUN OLEH:
RAFI RAMADHAN
109082500140
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Skema pemrosesan sekuensial merupakan konsep yang dirancang untuk memperkuat pemahaman terkait struktur kontrol perulangan. Penggunaan struktur kontrol ini memungkinkan penyelesaian berbagai kasus perulangan berdasarkan pola-pola yang telah diperkenalkan pada skema pemrosesan sekuensial. Dengan menggabungkan bentuk perulangan dan percabangan, banyak problem komputasi yang dapat diselesaikan datanya secara sekuensial.

Pola pertama yang umum digunakan adalah pembacaan data tanpa *marker* pada akhir rangkaian data. Pada pola ini, semua data yang diberikan pada saat masukan adalah data valid yang pasti diproses di dalam program. Data tersebut akan dibaca secara berurutan dan langsung dieksekusi oleh baris kode pemrosesan di dalam sistem perulangan.

Pola kedua adalah pembacaan data dengan menggunakan *marker* pada akhir rangkaian data. Skema ini mempersiapkan sebuah data khusus yang berfungsi untuk menghentikan perulangan. Pada dasarnya, semua data masukan adalah valid untuk diproses, kecuali data yang terakhir karena perannya hanya sebagai penghenti perulangan. Dalam penerapannya, selalu ada pengkondisian pada struktur perulangan yang akan secara terus-menerus mengecek apakah nilai data masukan tersebut merupakan sebuah *marker* atau bukan.

Pola ketiga berkaitan dengan kemungkinan terjadinya rangkaian data kosong pada skema ber-*marker*. Pola ini mengakomodasi kondisi di mana data pertama yang diberikan pada masukan sudah berupa *marker*, yang berarti tidak ada satu pun data yang valid atau disebut juga sebagai kasus kosong. Penyelesaian masalah ini dilakukan dengan cara menambahkan struktur kontrol pengkondisian atau percabangan sebelum masuk ke dalam perulangan utama untuk mengevaluasi apakah kasus kosong tersebut terjadi.

Pola keempat adalah penanganan elemen pertama yang perlu diproses tersendiri sebagai kasus khusus. Pada pendekatan ini, data pertama akan diproses terlebih dahulu secara khusus sebelum siklus perulangan dilakukan. Skema ini sangat bermanfaat pada kasus komputasi tertentu, seperti mencari nilai temperatur maksimum, agar program tidak perlu kesulitan bergantung pada asumsi inisialisasi nilai awal sembarang yang belum tentu valid

SOAL LATIHAN

1. Rerata Bilangan Real dengan Marker 9999

Source Code:

```
package main

import "fmt"

func main() {
    var x, total float64
    var count int

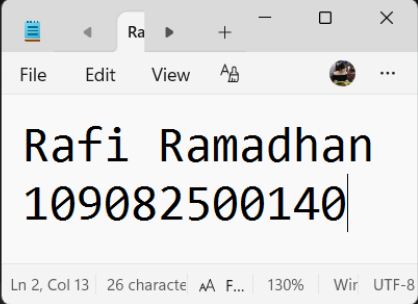
    fmt.Println("Masukkan bilangan (9999 untuk berhenti):")
    fmt.Scan(&x)

    for x != 9999 {
        total += x
        count++
        fmt.Scan(&x)
    }

    if count == 0 {
        fmt.Println("Tidak ada data")
    } else {
        fmt.Printf("Rerata = %.2f\n", total/float64(count))
    }
}
```

Output:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x, total float64
7     var count int
8
9     fmt.Println("Masukkan bilangan (9999 untuk berhenti):")
10    fmt.Scan(&x)
11
12    for x != 9999 {
13        total += x
14        count++
15        fmt.Scan(&x)
16    }
17
18    if count == 0 {
19        fmt.Println("Tidak ada data")
20    } else {
21        fmt.Printf("Rerata = %.2f\n", total/float64(count))
22    }
23 }
```



```
PS C:\alpro2> go run soal1.go
Masukkan bilangan (9999 untuk berhenti):
10
20
30
40
9999
Rerata = 25.00
PS C:\alpro2> 
```

Deskripsi Program:

Program ini digunakan untuk menghitung nilai rerata (rata-rata) dari sejumlah bilangan real yang dimasukkan oleh pengguna. Proses input dilakukan secara berulang hingga pengguna memasukkan nilai marker 9999 sebagai tanda akhir data. Setiap bilangan yang valid akan dijumlahkan dan dihitung banyaknya data, kemudian program menampilkan hasil rerata dari seluruh bilangan yang telah dimasukkan.

2. Pencarian String

Source Code:

```
package main

import "fmt"

func main() {
    var x, s string
    var n int

    fmt.Print("String yang dicari: ")
    fmt.Scan(&x)

    fmt.Print("Jumlah string: ")
    fmt.Scan(&n)

    jumlah := 0
    posisi := -1
```

```
for i := 1; i <= n; i++ {  
    fmt.Scan(&s)  
  
    if s == x {  
        jumlah++  
  
        if posisi == -1 {  
            posisi = i  
        }  
    }  
}  
  
fmt.Println("Ada?", jumlah > 0)  
  
if posisi != -1 {  
    fmt.Println("Posisi pertama:", posisi)  
} else {  
    fmt.Println("Tidak ditemukan")  
}  
  
fmt.Println("Jumlah kemunculan:", jumlah)  
fmt.Println("Minimal dua kali?", jumlah >= 2)  
}
```

Output:

The image shows a Go program in a code editor and its execution output in a terminal. The program is designed to find a specific string within a list of strings, count its occurrences, and check if it appears at least twice.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x, s string
7     var n int
8
9     fmt.Print("String yang dicari: ")
10    fmt.Scan(&x)
11
12    fmt.Print("Jumlah string: ")
13    fmt.Scan(&n)
14
15    jumlah := 0
16    posisi := -1
17
18    for i := 1; i <= n; i++ {
19        fmt.Scan(&s)
20
21        if s == x {
22            jumlah++
23
24            if posisi == -1 {
25                posisi = i
26            }
27        }
28    }
29 }
```

The terminal output shows the program being run with the input string 'apel' and 6 strings: 'mangga', 'apel', 'jeruk', 'apel', 'anggur', and 'melon'. The results are: 'Ada? true', 'Posisi pertama: 2', 'Jumlah kemunculan: 2', and 'Minimal dua kali? true'.

```
PS C:\alpro2> go run soal2.go
String yang dicari: apel
Jumlah string: 6
mangga
apel
jeruk
apel
anggur
melon
Ada? true
Posisi pertama: 2
Jumlah kemunculan: 2
Minimal dua kali? true
PS C:\alpro2>
```

Deskripsi Program:

Program ini digunakan untuk mencari keberadaan sebuah string tertentu dalam kumpulan string yang diberikan. Program akan menentukan apakah string yang dicari ditemukan atau tidak, posisi kemunculan pertama string tersebut, jumlah kemunculannya dalam data, serta memeriksa apakah string tersebut muncul sedikitnya dua kali dalam kumpulan data yang diinputkan.

3. Curah Hujan

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
```

```
rand.Seed(time.Now().UnixNano())

var n int

fmt.Print("Jumlah tetesan: ")
fmt.Scan(&n)

var A, B, C, D int

for i := 0; i < n; i++ {
    x := rand.Float64()
    y := rand.Float64()

    if x < 0.5 && y >= 0.5 {
        A++
    } else if x >= 0.5 && y >= 0.5 {
        B++
    } else if x < 0.5 && y < 0.5 {
        C++
    } else {
        D++
    }
}

fmt.Printf("Curah hujan daerah A: %.4f milimeter\n",
float64(A)*0.0001)
fmt.Printf("Curah hujan daerah B: %.4f milimeter\n",
float64(B)*0.0001)
fmt.Printf("Curah hujan daerah C: %.4f milimeter\n",
float64(C)*0.0001)
fmt.Printf("Curah hujan daerah D: %.4f milimeter\n",
float64(D)*0.0001)
}
```

Output:

```
soal3.go > main
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8
9 func main() {
10     rand.Seed(time.Now().UnixNano())
11
12     var n int
13
14     fmt.Print("Jumlah tetesan: ")
15     fmt.Scan(&n)
16
17     var A, B, C, D int
18
19     for i := 0; i < n; i++ {
20         x := rand.Float64()
21         y := rand.Float64()
22
23         if x < 0.5 && y >= 0.5 {
24             A++
25         } else if x >= 0.5 && y >= 0.5 {
26             B++
27         } else if x < 0.5 && y < 0.5 {
28             C++
29         } else {
30             D++
31         }
32     }
33
34     fmt.Println("Curah hujan daerah A: ", A)
35     fmt.Println("Curah hujan daerah B: ", B)
36     fmt.Println("Curah hujan daerah C: ", C)
37     fmt.Println("Curah hujan daerah D: ", D)
38 }
```

PS C:\alpro2> go run soal3.go
Jumlah tetesan: 1000
Curah hujan daerah A: 0.0267 milimeter
Curah hujan daerah B: 0.0257 milimeter
Curah hujan daerah C: 0.0252 milimeter
Curah hujan daerah D: 0.0224 milimeter
PS C:\alpro2> go run soal3.go
Jumlah tetesan: 1000000000
Curah hujan daerah A: 2499.7181 milimeter
Curah hujan daerah B: 2500.4774 milimeter
Curah hujan daerah C: 2499.9052 milimeter
Curah hujan daerah D: 2499.8993 milimeter
PS C:\alpro2> |

Deskripsi Program:

Program ini mensimulasikan jatuhnya tetesan hujan secara acak pada bidang berukuran 1×1 yang dibagi menjadi empat daerah, yaitu A, B, C, dan D. Setiap tetesan dihasilkan menggunakan koordinat acak dan akan dihitung berdasarkan daerah tempat jatuhnya. Setelah seluruh tetesan diproses, program mengonversi jumlah tetesan pada masing-masing daerah menjadi nilai curah hujan dalam satuan milimeter dan menampilkannya sebagai hasil simulasi.

4. Perhitungan Nilai π dengan Deret Leibniz

Source Code:

```
package main

import (
    "fmt"
    "math"
)

func main() {
```

```

var n int
var sum float64

fmt.Print("N suku pertama: ")
fmt.Scan(&n)

for i := 1; i <= n; i++ {
    suku := 1.0 / float64(2*i-1)

    if i%2 == 0 {
        suku = -suku
    }

    sum += suku
}

pi := 4 * sum
fmt.Printf("Hasil PI (%d suku) : %.10f\n", n, pi)

i := 1
for {
    s1 := 1.0 / float64(2*i-1)
    s2 := 1.0 / float64(2*(i+1)-1)

    if i%2 == 0 {
        s1 = -s1
    }

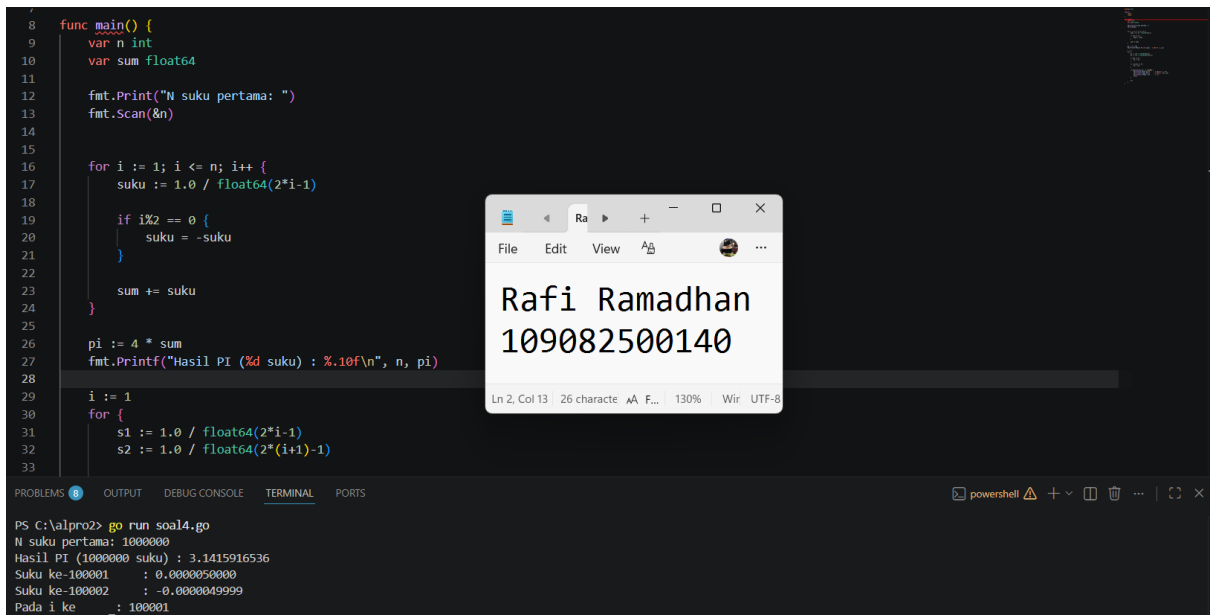
    if (i+1)%2 == 0 {
        s2 = -s2
    }

    if math.Abs(s1-s2) <= 0.00001 {
        fmt.Printf("Suku ke-%d      : %.10f\n", i, s1)
        fmt.Printf("Suku ke-%d      : %.10f\n", i+1, s2)
        fmt.Println("Pada i ke      :", i)
        break
    }

    i++
}
}

```

Output:



```
8 func main() {
9     var n int
10    var sum float64
11
12    fmt.Print("N suku pertama: ")
13    fmt.Scan(&n)
14
15
16    for i := 1; i <= n; i++ {
17        suku := 1.0 / float64(2*i-1)
18
19        if i%2 == 0 {
20            suku = -suku
21        }
22
23        sum += suku
24    }
25
26    pi := 4 * sum
27    fmt.Printf("Hasil PI (%d suku) : %.10f\n", n, pi)
28
29    i := 1
30    for {
31        s1 := 1.0 / float64(2*i-1)
32        s2 := 1.0 / float64(2*(i+1)-1)
33    }
```

PS C:\alpro2> go run soal4.go
N suku pertama: 1000000
Hasil PI (1000000 suku) : 3.1415916536
Suku ke-100001 : 0.0000050000
Suku ke-100002 : -0.0000049999
Pada i ke : 100001

Deskripsi Program:

Program ini digunakan untuk menghitung pendekatan nilai π menggunakan deret Leibniz, yaitu deret harmonik ganti yang terdiri dari suku positif dan negatif secara bergantian. Program menjumlahkan sejumlah suku tertentu yang diberikan pengguna, kemudian mengalikan hasilnya dengan empat untuk memperoleh nilai pendekatan π . Pada versi modifikasi, program akan berhenti ketika selisih dua suku berurutan sudah lebih kecil atau sama dengan batas toleransi yang ditentukan.

5. Simulasi Pizza dan Perhitungan π

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var inside int

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            inside++
        }
    }
}
```

```

    pi := 4.0 * float64(inside) / float64(n)

    fmt.Println("Topping pada Pizza:", inside)
    fmt.Printf("PI : %.10f\n", pi)
}

```

Output:

```

1  package main
2
3  import (
4      "fmt"
5      "math/rand"
6  )
7
8  func main() {
9      var n int
10     var inside int
11
12     fmt.Print("Banyak Topping: ")
13     fmt.Scan(&n)
14
15     for i := 0; i < n; i++ {
16         x := rand.Float64()
17         y := rand.Float64()
18
19         if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
20             inside++
21         }
22     }
23
24     pi := 4.0 * float64(inside) / float64(n)
25
26     fmt.Println("Topping pada Pizza:", inside)
27     fmt.Printf("PI : %.10f\n", pi)
28 }

```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\alpro2> go run soal5.go
Banyak Topping: 5000
Topping pada Pizza: 3880
PI : 3.1040000000

```

Deskripsi Program:

Program ini mensimulasikan penaburan topping pada sebuah wadah berbentuk persegi yang berisi pizza berbentuk lingkaran. Setiap topping direpresentasikan sebagai titik acak pada bidang koordinat. Program akan menghitung jumlah topping yang berada di dalam area pizza menggunakan persamaan lingkaran, kemudian menggunakan perbandingan jumlah topping di dalam pizza terhadap jumlah topping keseluruhan untuk memperkirakan nilai konstanta π dengan metode Monte Carlo.

DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2, Bab 6: Pengayaan (Modul 17: Skema Pemrosesan Sekuensial), School of Computing, Telkom University.